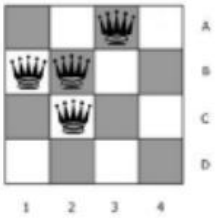
BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Phạm Hồng Thái
MSSV/ Lớp	3123410331

Bài 1. Bài toán 4-Queens và Hill Climbing

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Thử di chuyển từng quân hậu trên cột của nó (hoặc sang cột trống) để tìm trạng thái láng giềng có số xung đột h thấp nhất:

- **Sửa cột 2: Di chuyển hậu từ (C, 2) xuống (D, 2):**
  - Vị trí hậu mới: (B, 1), (B, 2), (D, 2), (A, 3).
  - Kiểm tra xung đột:
    - (B, 1) ngang (B, 2) (1)
    - (B, 2) chéo (D, 2) (1)

    $\rightarrow h = 2$.
- **Sửa cột 2: Di chuyển hậu từ (B, 2) lên (A, 2):**
  - Vị trí hậu mới: (B, 1), (A, 2), (C, 2), (A, 3).
  - Kiểm tra xung đột:
    - (A, 2) ngang (A, 3) (1)
    - (A, 2) dọc (C, 2) (1)

    $\rightarrow h = 2$.
- **Sửa cột 3: Di chuyển hậu từ (A, 3) xuống (D, 3):**
  - Vị trí hậu mới: (B, 1), (B, 2), (C, 2), (D, 3).
  - Kiểm tra xung đột:
    - (B, 1) ngang (B, 2) (1)
    - (B, 2) dọc (C, 2) (1)

    $\rightarrow h = 2$.
- **Sửa cột 2 & 4: Di chuyển hậu từ (C, 2) sang (D, 4):**

    - Vị trí hậu mới: (B, 1), (B, 2), (A, 3), (D, 4).
    - Kiểm tra xung đột:
        - (B, 1) ngang (B, 2) (1)
        - (B, 2) chéo (A, 3) (1)
        - (B, 2) chéo (D, 4) (1)
          → $h = 3$.

Kết luận: Trạng thái láng giềng có số xung đột nhỏ nhất tìm được là $h = 2$ (thấp hơn hẳn so với $h(current_state) = 4$).

Do đó, thuật toán sẽ chọn trạng thái láng giềng có h thấp nhất này để tiếp tục vòng lặp (ví dụ di chuyển quân hậu từ (C, 2) xuống (D, 2)).

- **Đáp án câu a:** Trạng thái sau 1 vòng lặp là:
    - Cột 1: **(B, 1)**
    - Cột 2: **(B, 2)**
    - Cột 2 (hoặc Cột 4): **(D, 2)** *(hoặc chọn di chuyển A3 → D3)*
    - Cột 3: **(A, 3)**

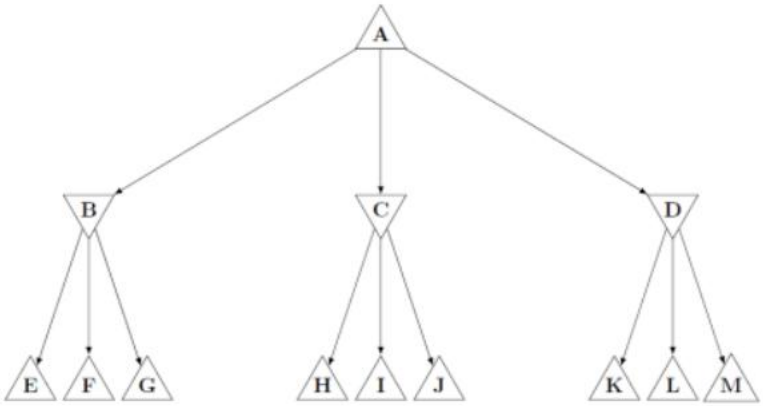
b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

- Thuật toán không dừng ngay ở trạng thái ban đầu mà sẽ tiếp tục di chuyển sang trạng thái láng giềng có $h = 2$.
- Sau một số vòng lặp tiếp theo, thuật toán sẽ bị kẹt tại một trạng thái có $h > 0$ mà mọi láng giềng xung quanh đều không thể giảm thêm xung đột. Vì không thể đạt tới $h = 0$ (nghiệm toàn cục), trạng thái cuối cùng dừng lại thuộc loại Cực trị cục bộ.

Đáp án câu b: **B) Cực trị cục bộ (local optimum)**

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

Gốc A là nút MAX, các con B, C, D là nút MIN.
Vì nút cha A chọn giá trị lớn nhất (MAX), góc nhìn tốt nhất của A đối với các cây con B, C, D là cây có giá trị đánh giá $f(s)$ cao nhất:

- $f(B) = 4$
- $f(C) = 14$
- $f(D) = 19$

Sắp xếp từ tốt nhất đến xấu nhất theo góc nhìn nút MAX A: D (19) → C (14) → B (4).
Chọn phương án: D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Duyệt theo thứ tự mới: D, sau đó đến C, cuối cùng đến B.

1. Xét cây con D (Nút MIN):
    - Duyệt con K: $f(K) = 23$ → Nút D cập nhật $v = min(\infty, 23) = 23$.
    - Duyệt con L: $f(L) = 20$ → Nút D cập nhật $v = min(23,20) = 20$.
    - Duyệt con M: $f(M) = 26$ → Nút D cập nhật $v = min(20,26) = 20$.
    - Giá trị trả về tại D = 20.

- Nút A (MAX) cập nhật $\alpha = max(-\infty, 20) = 20$.

2. Xét cây con C (Nút MIN, với $\alpha = 20$):
   - Duyệt con H: $f(H) = 17 \rightarrow$ Nút C cập nhật $v = min(\infty, 17) = 17$.
   - Vì $v = 17 \leq \alpha = 20$, điều kiện cắt tỉa $\beta \leq \alpha$ thỏa mãn → Cắt tỉa nhánh C.
   - Các nút con còn lại của C là I, J KHÔNG được thăm.
3. Xét cây con B (Nút MIN, với $\alpha = 20$):
   - Duyệt con E: $f(E) = 5 \rightarrow$ Nút B cập nhật $v = min(\infty, 5) = 5$.
   - Vì $v = 5 \leq \alpha = 20$, điều kiện cắt tỉa thỏa mãn → Cắt tỉa nhánh B.
   - Các nút con còn lại của B là F, G KHÔNG được thăm.

=> Kết luận các nút KHÔNG được thăm: I, J, F, G

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

☐ Có

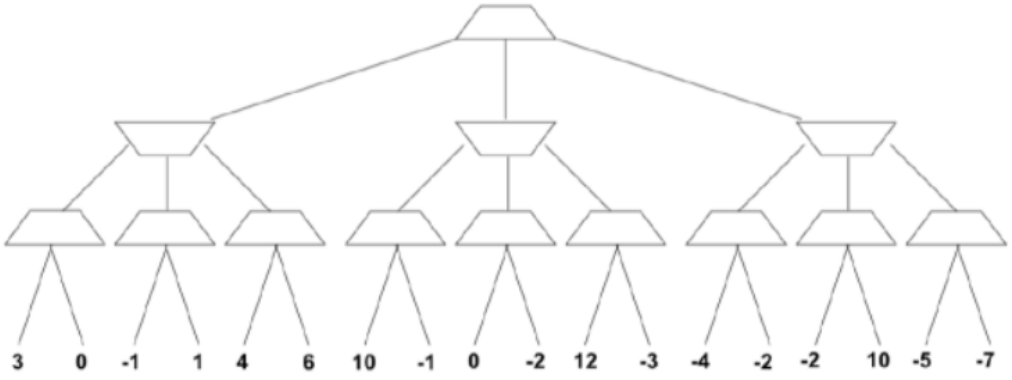
☐ Không

Chọn phương án: Không

- **Giải thích:** Hàm đánh giá $f(s)$ ở độ sâu trung gian chỉ là hàm ước lượng/tiên đoán chứ không phải giá trị Minimax thực sự của nút lá. Việc sắp xếp thứ tự duyệt dựa trên $f(s)$ có thể dẫn đến cắt tỉa sai các nhánh mà nếu duyệt hết sẽ cho giá trị Minimax tối ưu thực sự.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:
• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.
• Ghi giá trị được trả về tại mỗi nút trung gian.
• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.
• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Nhánh trái (Nút 1 - MIN):

- Thăm lá 1: $3 \rightarrow v_1 = 3$.
- Thăm lá 2: $0 \rightarrow v_1 = min(3,0) = 0$.
- Thăm lá 3: $-1 \rightarrow v_1 = min(0,-1) = -1$.
- Nút 1 trả về -1.
- Nút Gốc (MAX) cập nhật $\alpha = max(-\infty,-1) = -1$.

Nhánh giữa (Nút 2 - MIN, với $\alpha = -1$):

- Sub-node 2.1 (MAX):
  - Lá $1 \rightarrow v = 1$.
  - Lá $4 \rightarrow v = max(1,4) = 4$.
  - Trả về 4.
  - Nút 2 (MIN) có $v_2 = min(\infty, 4) = 4$.
- Sub-node 2.2 (MAX):
  - Lá $6 \rightarrow v = 6$.
  - Lá $10 \rightarrow v = max(6,10) = 10$.
  - Trả về $10 \rightarrow v_2 = min(4,10) = 4$.
- Sub-node 2.3 (MAX):
  - Lá $-1 \rightarrow v = -1$.
  - Lá $0 \rightarrow v = max(-1,0) = 0$.
  - Trả về $0 \rightarrow v_2 = min(4,0) = 0$.

- Nút 2 trả về 0.
- Nút Gốc (MAX) cập nhật $\alpha = max(-1,0) = 0$.

Nhánh phải (Nút 3 - MIN, với $\alpha = 0$):

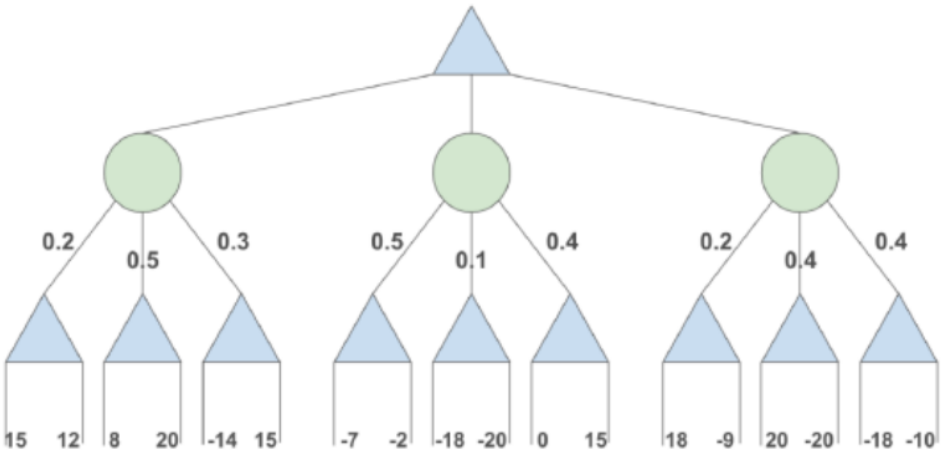
- Sub-node 3.1 (MAX):
  - Lá $-2 \rightarrow v = -2$.
  - Lá $12 \rightarrow v = max(-2,12) = 12$.
  - Trả về $12 \rightarrow v_3 = min(\infty, 12) = 12$.
- Sub-node 3.2 (MAX):
  - Lá $-3 \rightarrow v = -3$.
  - Lá $-4 \rightarrow v = max(-3,-4) = -3$.
  - Trả về $-3 \rightarrow v_3 = min(12,-3) = -3$.
  - Cắt tỉa: Vì $v_3 = -3 \leq \alpha = 0 \rightarrow$ Cắt toàn bộ Sub-node 3.3 (gồm các lá $-2,-2,10,-5,-7$).

Kết quả yêu cầu:

- Giá trị Minimax tại nút gốc: 0
- Nước đi tối ưu của MAX: Chọn nhánh giữa (Nút 2).
- Các nhánh bị cắt tỉa (đánh dấu X): Nhánh chứa các lá từ -2 đến -7 ở góc bên phải (Sub-node 3.3).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

1. Tầng nút MAX (đáy cây):
Các nút MAX chọn giá trị lớn nhất (*max*) từ các lá của chúng:

- **Cụm 1 (Trái):**
  - Nút MAX 1: $max(15,12) = 15$
  - Nút MAX 2: $max(8,20) = 20$
  - Nút MAX 3: $max(-14,15) = 15$
- **Cụm 2 (Giữa):**
  - Nút MAX 4: $max(-7,-2) = -2$
  - Nút MAX 5: $max(-18,-20) = -18$
  - Nút MAX 6: $max(0,15) = 15$
- **Cụm 3 (Phải):**
  - Nút MAX 7: $max(18,-9) = 18$
  - Nút MAX 8: $max(20,-20) = 20$
  - Nút MAX 9: $max(-18,-10) = -10$

2. Tầng Chance Nodes (hình tròn):
Giá trị tại Chance Node = $\sum \quad (Xácsuất \times GiátrịnútMAXcon)$

- **Chance Node 1 (Trái):**
  - $V_{C1} = (0.2 \times 15) + (0.5 \times 20) + (0.3 \times 15) = 3 + 10 + 4.5 = 17.5$
- **Chance Node 2 (Giữa):**
  - $V_{C2} = (0.5 \times -2) + (0.1 \times -18) + (0.4 \times 15) = -1.0 - 1.8 + 6.0 = 3.2$
- **Chance Node 3 (Phải):**
  - $V_{C3} = (0.2 \times 18) + (0.4 \times 20) + (0.4 \times -10) = 3.6 + 8.0 - 4.0 = 7.6$

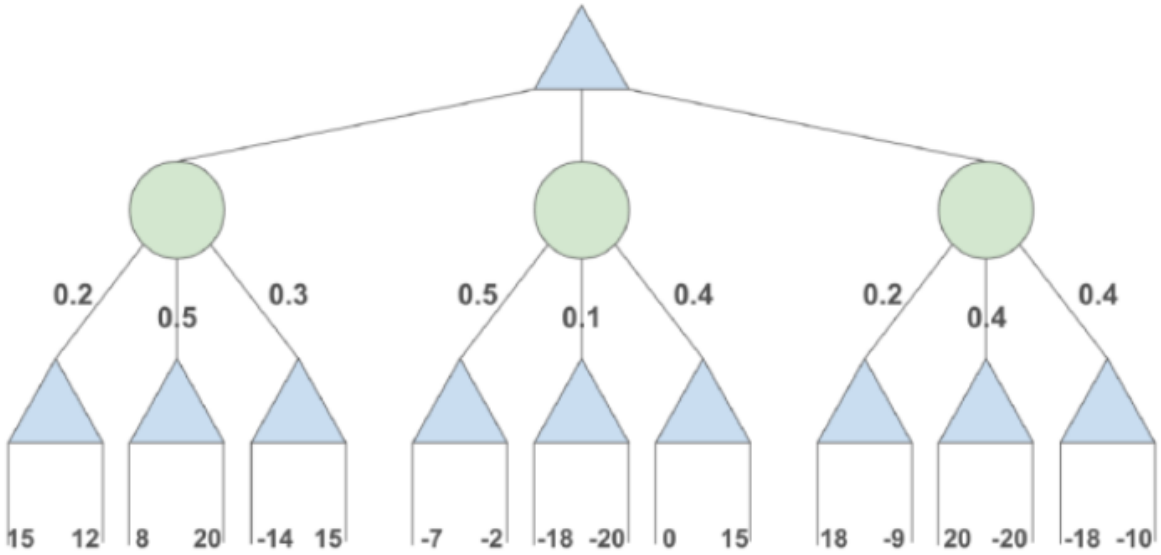
3. Nút Gốc MAX (tam giác):

$$Gốc = max(V_{C1}, V_{C2}, V_{C3}) = max(17.5,3.2,7.6) = 17.5$$

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

- **Trả lời:** Không có nút nào có thể bị cắt tỉa.
- **Giải thích:** Trong cây Expectimax, giá trị tại chance node là trung bình có trọng số của tất cả các con của nó. Nếu không có giới hạn (bound) về giá trị lá, một lá chưa được thăm có thể nhận giá trị cực kỳ lớn ($+\infty$ hoặc $-\infty$), làm thay đổi hoàn toàn giá trị kỳ vọng của chance node đó. Vì vậy, thuật toán buộc phải duyệt hết tất cả các nhánh để tính chính xác.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Với $V_{min} = -20$ và $V_{max} = 20$:

1. **Duyệt Cụm 1 (Trái):** Tính được $V_{C1} = 17.5$.
    - Nút gốc MAX nhận giá trị tạm thời ≥ 17.5.
2. **Duyệt Cụm 2 (Giữa):**
    - Giới hạn trên tốt nhất có thể đạt được của V_{C2} nếu các nhánh còn lại đạt giá trị tối đa ($+20$):
        - Sau khi tính Nút MAX 4 $= -2$: $V_{C2} \leq (0.5 \times -2) + (0.1 \times 20) + (0.4 \times 20) = -1 + 2 + 8 = 9.0$
    - Do $9.0 < 17.5$ (giá trị hiện tại của gốc), V_{C2} chắc chắn không thể vượt qua V_{C1}.
    - Cắt tỉa: Cắt toàn bộ các nhánh còn lại của Cụm 2 (Nút MAX 5 và Nút MAX 6).
3. **Duyệt Cụm 3 (Phải):**

- Sau khi tính Nút MAX 7 = 18:
  - Giới hạn trên của $V_{C3} \leq (0.2 \times 18) + (0.4 \times 20) + (0.4 \times 20) = 3.6 + 8 + 8 = 19.6$.
  - Vì $19.6 > 17.5$, chưa thể cắt tỉa ngay.
- Sau khi tính Nút MAX 8 = 20:
  - $V_{C3} \leq (0.2 \times 18) + (0.4 \times 20) + (0.4 \times 20) = 19.6$. Cần tính tiếp Nút MAX 9.
- Tính Nút MAX 9 = $-10 \rightarrow V_{C3} = 7.6$.

Các nút/nhánh bị cắt tỉa: Nút MAX 5 và Nút MAX 6 (gồm các lá $-18, -20, 0, 15$).

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Tiêu chí	Forward Checking (FC)	AC-3 (Arc Consistency)
Phạm vi lan truyền ràng buộc	Cục bộ (1 bước): Chỉ kiểm tra và loại bỏ các giá trị trong miền giá trị của các biến chưa gán *có liên kết trực tiếp* với biến vừa được gán.	Toàn cục (Đa bước): Lan truyền sự thay đổi qua toàn bộ mạng lưới ràng buộc (queue các cung). Nếu miền giá trị của Y bị thu hẹp, nó tiếp tục kiểm tra lại các biến X liên kết với Y.
Thời điểm thực hiện	Thực hiện ngay sau mỗi bước gán giá trị cho một biến mới.	Có thể chạy tiền xử lý trước khi tìm kiếm, hoặc chạy kết hợp sau mỗi bước gán (MAC - Maintaining Arc Consistency).
Chi phí tính toán	Thấp: Độ phức tạp nhỏ, thực hiện nhanh ở mỗi bước.	Cao hơn: Chi phí tính toán lớn hơn cho mỗi bước gán, nhưng giúp phát hiện mâu thuẫn sớm hơn rất nhiều.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV) - Biến còn ít giá trị nhất

- **Lý do sử dụng (Chiến lược Fail-First):** MRV chọn biến có ít giá trị hợp lệ còn lại nhất trong miền giá trị để tiến hành gán trước.
- **Tác dụng:** Giúp phát hiện ra nhánh cụt (mâu thuẫn/không có lời giải) sớm nhất có thể, từ đó tỉa bớt cây tìm kiếm ngay lập tức mà không phải tốn công duyệt sâu vào các nhánh thất bại.

ii) Least Constraining Value (LCV) - Giá trị ít gây ràng buộc nhất

- **Lý do sử dụng (Chiến lược Fail-Last):** Sau khi đã chọn biến, LCV ưu tiên chọn giá trị nào *ít làm mất đi các lựa chọn* của các biến chưa gán còn lại.
- **Tác dụng:** Tối đa hóa cơ hội tìm thấy lời giải ngay ở lần thử đầu tiên, giúp thuật toán nhanh chóng vạch ra con đường dẫn tới lời giải thành công mà ít phải quay lui (backtrack).

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung
giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa
các ràng buộc sau:
• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.
• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C.
• Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

- **Tập biến (Variables):** $X = \{A, B, C, D, E\}$.
- **Miền giá trị (Domains):** Với mỗi biến $X_i \in X$, miền giá trị là $D_i = \{1,2,3,4\} \times \{Quốc tế, Nội địa\}$. *(Hoặc ký hiệu cặp (khung giờ, đường băng), ví dụ: (1, INT), (1, DOM), ...)*
- **Ràng buộc đơn/nguyên (Unary constraints):**
  - $B \in \{(1, INT), (1, DOM)\}$ (Máy bay B ở khung giờ 1).
  - $D \in \{(3, INT), (3, DOM), (4, INT), (4, DOM)\}$ (Máy bay D từ khung giờ 3 trở đi).
  - $A \in \{(1, INT), (1, DOM), (2, INT), (2, DOM)\}$ (Máy bay A tối đa đến khung giờ 2).
- **Ràng buộc nhị phân (Binary constraints):**
  - $Time(D) < Time(C)$ (Máy bay D hạ cánh trước khi C cất cánh).

- Với mọi cặp (X_i, X_j) phân biệt: $\neg(Time(X_i) = Time(X_j) \wedge Runway(X_i) = Runway(X_j))$ (Không cùng khung giờ trên cùng đường băng).

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Vì loại đường băng đã được cố định bởi bản chất chuyến bay:

- Chuyến bay quốc tế (A, B, C): Dùng đường băng Quốc tế.
- Chuyến bay nội địa (D, E): Dùng đường băng Nội địa.

Miền giá trị chỉ còn chứa khung giờ: $D_A = D_B = D_C = D_D = D_E = \{1,2,3,4\}$.

Đồ thị ràng buộc nhị phân (Constraint Graph):

- Nhóm Quốc tế: Các đỉnh A, B, C nối với nhau bằng cạnh ≠ (khác khung giờ vì dùng chung đường băng Quốc tế).
- Nhóm Nội địa: Các đỉnh D, E nối với nhau bằng cạnh ≠ (khác khung giờ vì dùng chung đường băng Nội địa).
- Giữa hai nhóm: Cạnh định hướng giữa D và C biểu diễn ràng buộc $D < C$.

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

1. **Sau khi áp dụng Unary constraints:**
    - $D_A = \{1,2\}$ (A ≤ 2)
    - $D_B = \{1\}$ (B = 1)
    - $D_C = \{1,2,3,4\}$
    - $D_D = \{3,4\}$ (D ≥ 3)
    - $D_E = \{1,2,3,4\}$
2. **Áp dụng Arc Consistency (AC-3):**
    - Xét cung (B, A): Vì $B = 1$, để $A \neq B$ thì A không thể nhận giá trị $1 \rightarrow D_A = \{2\}$
    - Xét cung (B, C): C ≠ B => C ≠ 1 → D_C= {2, 3, 4}.
    - Xét cung (A, C): Vì $A = 2$, để $C \neq A$ thì C không thể nhận giá trị $2 \rightarrow D_C = \{3,4\}$.
    - Xét cung (D, C) với ràng buộc $D < C$:
        - Nếu $C = 3$, D có thể bằng 3 hoặc 4? Không thỏa $D < C$. Để có $D < C$ với $D \in \{3,4\}$, C phải lớn hơn D.
        - Nếu D = 3 => C phải bằng 4. Do đó $D_C = \{4\}$ và $D_D = \{3\}$.
    - Xét cung (D, E): Vì $D = 3$, $E \neq 3 \rightarrow D_E = \{1,2,4\}$.

Miền giá trị còn lại của các biến:

- $D_A = \{2\}$
- $D_B = \{1\}$
- $D_C = \{4\}$
- $D_D = \{3\}$
- $D_E = \{1,2,4\}$

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Thực hiện theo thuật toán:

- **Áp dụng Unary:** $D_A = \{1,2\}, D_B = \{1\}, D_C = \{1,2,3,4\}, D_D = \{3,4\}, D_E = \{1,2,3,4\}$.
- **Chọn biến theo MRV (kích thước miền nhỏ nhất):** B có kích thước 1.
  - Gán $(\boldsymbol{B}, \mathbf{1})$.
    - Forward Checking: Loại 1 khỏi A và C.
    - Miền còn lại: $D_A = \{2\}, D_C = \{2,3,4\}, D_D = \{3,4\}, D_E = \{1,2,3,4\}$.
- **Chọn biến tiếp theo (MRV):** A có kích thước 1. 2. Gán $(\boldsymbol{A}, \mathbf{2})$.
  - Forward Checking: Loại 2 khỏi C.
  - Miền còn lại: $D_C = \{3,4\}, D_D = \{3,4\}, D_E = \{1,2,3,4\}$.
- **Chọn biến tiếp theo (MRV):** C và D hòa (đều có kích thước 2). Theo thứ tự ký tự nhỏ hơn → Chọn C.
  - Giá trị nhỏ hơn trước → Thử $C = 3$.
  - Gán $(\boldsymbol{C}, \mathbf{3})$.
    - Forward Checking với D < C => D < 3 => D ∈ {1, 2}. Nhưng miền của D là $\{3,4\}$, không có giá trị nào < 3.
    - → **Dead end tại D!**
    - Backtrack: Hủy gán $(C, 3)$.
  - Gán $(\boldsymbol{C}, \mathbf{4})$.
    - Forward Checking với D < C => D < 4 => D = 3.
    - Miền còn lại: $D_D = \{3\}, D_E = \{1,2,3,4\}$.
- **Chọn biến tiếp theo (MRV):** D có kích thước 1. 5. Gán $(\boldsymbol{D}, \mathbf{3})$.
  - Forward Checking: Loại 3 khỏi E.

- Miền còn lại: $D_E = \{1,2,4\}$.

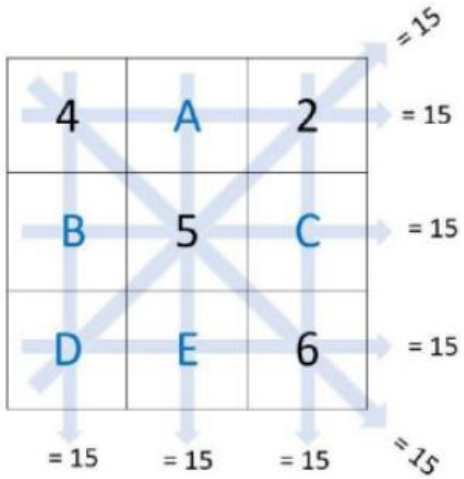
- **Chọn biến tiếp theo (MRV):** E. 6. Gán $(\boldsymbol{E}, \mathbf{1})$ (giá trị nhỏ nhất).

Thứ tự các cặp (biến, giá trị) được gán:

1. (B, 1)
2. (A, 2)
3. (C, 3)
4. (C, 4)
5. (D, 3)
6. (E, 1)

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n²}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Tập biến: $X = \{A, B, C, D, E\}$.
Miền giá trị: Các số chưa được dùng trong tập {1,2,3,4,5,6,7,8,9}. Các số đã biết: 4,2,5,6. => $D_A = D_B = D_C = D_D = D_E$ = {1, 3, 7, 8, 9}.
Ràng buộc:

- **AllDiff:** $alldiff(A, B, C, D, E)$ (các số phân biệt).
- **Tổng các hàng, cột, đường chéo = 15:**

  - Hàng 1: 4 + A + 2 = 15 => A = 9.
  - Hàng 2: B + 5 + C = 15 => B + C = 10.
  - Hàng 3: D + E + 6 = 15 => D + E = 9.

- Cột 1: 4 + B + D = 15 => B + D = 11.
- Cột 2: A + 5 + E = 15 => A + E = 10.
- Cột 3: 2 + C + 6 = 15 => C = 7.

b) Đồ thị ràng buộc nhị phân (bỏ qua alldiff)

Các cạnh ràng buộc nhị phân trực tiếp dựa trên phương trình tổng:

- Đỉnh **A** nối với **E** ($A + E = 10$)
- Đỉnh **B** nối với **C** ($B + C = 10$)
- Đỉnh **B** nối với **D** ($B + D = 11$)
- Đỉnh **D** nối với **E** ($D + E = 9$)

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Từ hàng 1: 4 + A + 2 = 15 => A = 9. Do đó $D_A = \{9\}$.
Từ cột 3: 2 + C + 6 = 15 => C = 7. Do đó $D_C = \{7\}$.
Áp dụng AC-3 lên các cạnh:

- Từ $A + E = 10$ với A = 9 => E = 1 => $D_E = \{1\}$.
- Từ $B + C = 10$ với C = 7 => B = 3 => $D_B = \{3\}$.
- Từ $D + E = 9$ với E = 1 => D = 8 => $D_D = \{8\}$.
- Kiểm tra lại $B + D = 3 + 8 = 11$ (Thỏa mãn!).

Nghiệm duy nhất của Ma trận Magic Square:

- $A = 9$
- $B = 3$
- $C = 7$
- $D = 8$
- $E = 1$

Ma trận đầy đủ thu được:

4	9	2
3	5	7
8	1	6

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Chọn biến đầu tiên (MRV):

- $Ellyse$ có 2 giá trị (nhỏ nhất) $\implies$ Chọn $Ellyse$.
- Chọn giá trị (LCV cho $Ellyse$):
  - Thử màu Đỏ: Loại màu Đỏ khỏi AJ và $Sunny$ (ảnh hưởng 2 biến).
  - Thử màu Hồng: Loại màu Hồng khỏi $Sunny$ (ảnh hưởng 1 biến).
  - LCV chọn giá trị ít gây hạn chế nhất $\implies$ Chọn màu Hồng.
- Gán: $(Ellyse, Hồng)$.
- Miền còn lại: $D_{AJ} = \{Đỏ, Vàng, Xanhlá\}$, $D_{Sunny} = \{Đỏ, Vàng\}$.

Chọn biến thứ hai (MRV):

- $Sunny$ có 2 giá trị còn lại (ít hơn AJ có 3) $\implies$ Chọn $Sunny$.
- Chọn giá trị (LCV cho $Sunny$):
  - Thử màu Đỏ: Loại Đỏ khỏi AJ (ảnh hưởng 1 biến).
  - Thử màu Vàng: Loại Vàng khỏi AJ (ảnh hưởng 1 biến).
  - Hòa LCV → Phá hòa theo thứ tự cầu vồng chọn màu Đỏ.
- Gán: $(Sunny, Đỏ)$.
- Miền còn lại: $D_{AJ} = \{Vàng, Xanhlá\}$.

Chọn biến thứ ba (MRV):

- Chọn AJ.
- Phá hòa theo thứ tự cầu vồng chọn màu Vàng.
- Gán: $(AJ, Vàng)$.

Thứ tự gán thành công:

1. (Ellyse, Hồng)

2. (Sunny, Đỏ)
3. (AJ, Vàng)

- **Số lần backtrack: 0 lần**.

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

- **Maximum Remaining Values:** Chọn biến có **nhiều** giá trị nhất trong miền.
- **Most Constraining Value:** Chọn giá trị gây hạn chế nhiều nhất cho các biến khác.

1. **Bước 1:**
   - AJ và $Sunny$ hòa với 3 giá trị. Theo thứ tự bảng chữ cái chọn AJ.
   - Chọn giá trị (Most Constraining):
     - Màu Đỏ bị trùng với cả $Sunny$ và $Ellyse$ (ảnh hưởng 2 biến) => Chọn màu Đỏ.
   - Gán: $(AJ, \text{Đỏ})$.
   - Miền còn lại: $D_{Sunny} = \{Vàng, Hồng\}$, $D_{Ellyse} = \{Hồng\}$.
2. **Bước 2:**
   - $Sunny$ có 2 giá trị còn lại (> $Ellyse$ có 1) $\implies$ Chọn $Sunny$.
   - Chọn giá trị (Most Constraining):
     - Màu Hồng bị trùng với $Ellyse$ (ảnh hưởng 1 biến) $\implies$ Chọn màu Hồng.
   - Gán: $(Sunny, Hồng)$.
   - Miền còn lại: D_{Ellyse} = Rỗng => Dead end tại $Ellyse$!
   - Backtrack: Hủy gán $(Sunny, Hồng)$, chuyển sang giá trị tiếp theo $(Sunny, Vàng)$.
3. **Bước 3:**
   - Gán lại $(Sunny, Vàng)$.
   - Miền còn lại: $D_{Ellyse} = \{Hồng\}$.
   - Gán $(Ellyse, Hồng)$.

- Số lần backtrack: 1 lần.
- So sánh & Giải thích:
  - Số lần backtrack tăng lên (từ 0 lên 1).
  - Giải thích: Heuristic MRV + LCV tiêu chuẩn được thiết kế theo nguyên lý *"Fail-First, Fail-Fast"* (MRV chọn biến dễ thất bại nhất trước để xử lý) và *"Fail-Last"* (LCV chọn giá trị dễ thành công nhất) giúp tối ưu đường đi. Ngược lại, Maximum Remaining Values và Most Constraining Value cố tình chọn con đường khó nhất và ép các biến khác vào ngõ hẹp, dẫn tới việc gặp mâu thuẫn (dead end) và phải backtrack.